Ingegneria del Software

Progetto DietiEstates25

Testing e Valutazione dell'Usabilita

Sezione di documentazione per la consegna di progetto

Gruppo:	DietiEstates25
Artefatti analizzati:	suite Jest backend, documentazione di progetto, UC2, UC4
Versione documento:	30 marzo 2026

Il layout del presente documento e stato reso coerente con i PDF gia presenti nella cartella Documentazione: frontespizio dedicato, indice, sezioni numerate e tabelle sintetiche per favorire una consegna omogenea.

Indice

1 Obiettivo e fonti **1**

2 Testing xUnit **1**
2.1 Tecnologia e impostazione della suite . . . 1
2.2 Metodi selezionati . . . 2
2.3 Strategia di progettazione dei test . . . 4
2.4 Analisi dei quattro metodi testati . . . 4
2.4.1 `OffertaDAO.updateStatoOfferta(idOfferta, nuovoStato)` . . . 4
2.4.2 `UtenteDAO.assignUserToAgency(idUtente, idAgenzia)` . . . 4
2.4.3 `OffertaDAO.createOfferta(idImmobile, idUtente, prezzoOfferto)` 5
2.4.4 `getNearbyPlaces(lat, lon, radius?)` . . . 5
2.5 Riepilogo della copertura progettata . . . 6
2.6 Allineamento completo con Test-Doc . . . 6
2.6.1 Tipologia di test: White box (dettaglio operativo) . . . 6
2.6.2 Metodi testati (firma e tipo) . . . 7
2.6.3 Dettaglio metodo 1: `updateStatoOfferta` . . . 7
2.6.4 Dettaglio metodo 2: `assignUserToAgency` . . . 7
2.6.5 Dettaglio metodo 3: `createOfferta` . . . 8
2.6.6 Dettaglio metodo 4: `getNearbyPlaces` . . . 8
2.6.7 Riepilogo criteri strutturali (tabella Test-Doc) . . . 9
2.6.8 File e configurazione . . . 9
2.7 Esito dell'esecuzione reale della suite . . . 9

3 Valutazione dell'usabilita **10**
3.1 Obiettivi e collegamento ai requisiti . . . 10
3.2 Expert review / inspection . . . 10
3.2.1 Metodo adottato . . . 10
3.2.2 Checklist di review . . . 10
3.2.3 Esiti e priorita di intervento . . . 11
3.3 Esperimento con utenti reali o potenziali . . . 11
3.3.1 Soggetti reclutati . . . 11
3.3.2 Procedura sperimentale . . . 12
3.3.3 Task sperimentali . . . 12
3.3.4 Metriche osservate . . . 12
3.3.5 Survey post-esperimento . . . 12
3.3.6 Risultati . . . 13
3.3.7 Discussione dei risultati . . . 14

4 Documentazione del processo di sviluppo **14**
4.1 Versionamento . . . 14
4.1.1 GitHub . . . 14
4.1.2 Statistiche del repository . . . 15
4.2 Codice sorgente e containerizzazione . . . 16
4.2.1 Dockerfile backend . . . 16
4.2.2 Dockerfile frontend . . . 17
4.2.3 docker-compose.yml e file di build automatica . . . 17

5 Conclusioni **18**

1 Obiettivo e fonti

Questo documento raccoglie la documentazione necessaria per il punto *Testing e valutazione dell'usabilita* del progetto DietiEstates25. Il contenuto e stato costruito a partire dagli artefatti realmente presenti nel repository e dai risultati prodotti dalla suite di test backend.

L'obiettivo e duplice:

- documentare il codice xUnit per quattro metodi non banali, descrivendo la strategia di progettazione dei test con classi di equivalenza e criteri di copertura;
- documentare una valutazione di usabilita completa, composta da expert review sistematica e da un esperimento con utenti reali o potenziali corredato da survey e discussione dei risultati.

Le fonti utilizzate sono le seguenti:

- `dietiestates-backend/test/backend.test.ts`;
- `dietiestates-backend/jest.config.ts`;
- `dietiestates-backend/package.json`;
- `Progettazione/Test-Doc.md`;
- `Progettazione/Requisiti.md`;
- `Progettazione/UC2 Caricamento Immobili.md`;
- `Progettazione/UC4 Gestione Offerte su.md`;
- i PDF gia presenti nella cartella `Documentazione`, usati come riferimento per struttura e tono editoriale.

2 Testing xUnit

2.1 Tecnologia e impostazione della suite

La suite di unit test del backend e implementata con **Jest** in ambiente Node.js, tramite preset `ts-jest`. Dall'analisi della configurazione emergono le seguenti scelte:

- esecuzione in ambiente `node`;
- localizzazione della suite nella cartella `test/`;
- raccolta della coverage sui file `src/**/*.ts`;
- esclusione di `src/server.ts` e dell'intera cartella `src/config/` dal calcolo della copertura.

Lo script effettivamente usato per l'esecuzione e il seguente:

```
npm test
// equivalente a: jest --runInBand --detectOpenHandles
```

La scelta di Jest e coerente con un'impostazione xUnit: ogni metodo viene testato in modo isolato, con fixture leggere, mocking delle dipendenze esterne e asserzioni mirate sul comportamento osservabile.

2.2 Metodi selezionati

La suite attuale copre quattro metodi che rispettano il requisito di essere non banali e di ricevere almeno due parametri formali. La Tabella 1 ne riassume la natura.

#	Metodo	Modulo	Motivazione della scelta
1	`updateStatoOfferta(idOfferta, nuovoStato)`	`OffertaDAO`	Aggiornamento persistente + mapping DTO con conversioni di tipo
2	`assignUserToAgency(idUtente, idAgenzia)`	`UtenteDAO`	UPDATE con WHERE condizionato su ruolo e gestione null
3	`createOfferta(idImmobile, idUtente, prezzoOfferto)`	`OffertaDAO`	Inserimento con valori di default di dominio e normalizzazione del record DB
4	`getNearbyPlaces(lat, lon, radius?)`	`utils/geoapify`	Chiamate HTTP multiple, lettura variabili d'ambiente e aggregazione risultati

Tabella 1: Metodi non banali coperti dai test xUnit backend.

2.3 Strategia di progettazione dei test

La progettazione dei test adotta un approccio **white box**. Tale scelta e giustificata dal fatto che i metodi verificati sono concentrati nel backend e dipendono da dettagli implementativi noti: query SQL costruite dinamicamente, gestione di campi opzionali, mapping di record provenienti dal database e integrazione con servizi esterni.

Le tecniche principali impiegate sono:

- **classi di equivalenza**, per partizionare il dominio degli input in insiemi significativi di casi validi e non validi;
- **statement coverage**, per verificare il percorso nominale di esecuzione e il corretto completamento delle operazioni previste;
- **branch coverage**, nei metodi con diramazioni esplicite o condizioni di errore;
- **boundary analysis**, dove sono presenti parametri opzionali o valori di default rilevanti;
- **mocking delle dipendenze esterne**, tramite mock di PostgreSQL e Axios per eliminare dipendenze da rete e database reale.

Questa impostazione consente test deterministici, veloci e ripetibili, caratteristica essenziale in una pipeline di verifica continua.

2.4 Analisi dei quattro metodi testati

2.4.1 `OffertaDAO.updateStatoOfferta(idOfferta, nuovoStato)`

Il metodo aggiorna lo stato di un'offerta nel database e converte la riga restituita nel corrispondente DTO applicativo. La logica non e banale perche unisce persistenza, parametrizzazione SQL e normalizzazione di campi numerici provenienti dal DB come stringhe.

Classi di equivalenza.

- CE1: offerta esistente e nuovo stato valido, ad esempio `(42, 'Accettata')`;
- CE2: offerta inesistente, con query che non restituisce righe;
- CE3: offerta esistente con campi DB da convertire, ad esempio `prezzoofferto = '200000'`.

Copertura adottata. Per questo metodo si applica soprattutto **statement coverage** sul percorso nominale, con verifica esplicita della query emessa e del mapping del risultato.

Evidenza dal test.

```
const updated = await updateStatoOfferta(42, 'Accettata');

expect(pool.query).toHaveBeenCalledWith(
  'UPDATE offerta SET stato = $1 WHERE idofferta = $2 RETURNING *',
  ['Accettata', 42]
);
expect(updated.prezzoOfferto).toBe(200000);
```

2.4.2 `UtenteDAO.assignUserToAgency(idUtente, idAgenzia)`

Il metodo assegna un utente (Agente o Supporto) a un'agenzia, eseguendo una `UPDATE` con clausola `WHERE` condizionata sul ruolo. La non banalita risiede nella verifica del ruolo e nella possibilita che l'utente non sia assegnabile (caso in cui il metodo restituisce `null`).

Classi di equivalenza.

- CE1: utente con ruolo Agente o Supporto, assegnazione riuscita;
- CE2: utente inesistente o con ruolo non consentito, restituisce `null`.

Copertura adottata. Qui e rilevante la **branch coverage**: il test copre sia il ramo nominale (assegnazione avvenuta), sia il ramo di mancata assegnazione (`null`). Viene inoltre verificata la correttezza della query e dei parametri passati.

Evidenza dal test.

```
const updated = await assignUserToAgency(7, 3);
expect(updated?.idUtente).toBe(7);
expect(updated?.idAgenzia).toBe(3);

const result = await assignUserToAgency(999, 3);
expect(result).toBeNull();
```

2.4.3 `OffertaDAO.createOfferta(idImmobile, idUtente, prezzoOfferto)`

Il metodo incapsula la creazione di una nuova offerta, valorizzando automaticamente parametri di dominio come `offertaManuale = false` e `idOffertaOriginale = null`. Anche in questo caso e presente mapping dal formato DB al DTO applicativo.

Classi di equivalenza.

- CE1: creazione standard con input validi e positivi;
- CE2: offerta originale assente, che deve risultare `null`;
- CE3: valori numerici restituiti dal DB come stringhe, da convertire.

Copertura adottata. Si applica **statement coverage** sul percorso nominale e controllo esplicito del payload inviato all'`INSERT`.

Evidenza dal test.

```
const offerta = await createOfferta(50, 20, 90000);

expect(pool.query).toHaveBeenCalledWith(
  expect.stringContaining('INSERT INTO offerta'),
  [50, 20, 90000, false, null]
);
expect(offerta.offertaManuale).toBe(false);
```

2.4.4 `getNearbyPlaces(lat, lon, radius?)`

Il metodo interroga il servizio esterno Geoapify per piu categorie di punti di interesse e aggrega i risultati. La funzione e non banale perche legge una variabile d'ambiente, invoca tre richieste HTTP distinte e deve gestire il caso di chiave assente.

Classi di equivalenza.

- CE1: chiave API presente e risposte valide dal servizio esterno;
- CE2: chiave API assente, con eccezione immediata;
- CE3: una o piu categorie senza risultati;
- CE4: raggio passato esplicitamente e diverso dal default.

Copertura adottata. Il metodo richiede **branch coverage** per coprire il ramo di errore e il flusso nominale. E inoltre presente una forma di **boundary analysis** sul parametro opzionale `radius`.

Evidenza dal test.

```
const places = await getNearbyPlaces(45, 9, 500);

expect(axios.get).toHaveBeenCalledTimes(3);
expect(places).toHaveLength(2);
await expect(getNearbyPlaces(45, 9)).rejects.toThrow('Chiave Geoapify mancante');
```

2.5 Riepilogo della copertura progettata

Metodo	Equivalenza	Statement	Branch	Note
`updateStatoOfferta`	Si	Si	Parziale	Verifica query e mapping DTO
`assignUserToAgency`	Si	Si	Si	Coperto anche il caso di mancata assegnazione
`createOfferta`	Si	Si	Parziale	Verifica valori di default di dominio
`getNearbyPlaces`	Si	Si	Si	Coperti chiave assente e raggio esplicito

Tabella 2: Strategie di copertura adottate nella suite xUnit.

2.6 Allineamento completo con Test-Doc

Questa sezione riporta in forma esplicita tutti gli elementi presenti in `Progettazione/Test-Doc.md`, in modo che il presente file `.tex` contenga integralmente il contenuto richiesto.

2.6.1 Tipologia di test: White box (dettaglio operativo)

Tutti i test adottati sono **white box**. In particolare:

- e noto il testo delle query SQL prodotte (verifica di UPDATE parametrizzate);
- e noto il comportamento di `insertOfferta` sui parametri `offertaManuale` e `offertaOriginaleId -> null`;
- e noto il flusso di `getNearbyPlaces` (3 categorie fisse e lettura di `process.env.GEOAPIFY_KEY` a runtime);
- sono noti i branch interni di `assignUserToAgency` (check su `result.rows[0]` e clausola `Ruolo IN (...)`).

Il mocking di `pool` (PostgreSQL) e `axios` isola ogni unita dalle dipendenze esterne, rendendo i test deterministici e ripetibili.

2.6.2 Metodi testati (firma e tipo)

#	Metodo	Modulo	Firma	Tipo
1	`updateStatoOfferta`	`OffertaDAO`	`(idOfferta: number, nuovoStato: string)`	White box
2	`assignUserToAgency`	`UtenteDAO`	`(idUtente: number, idAgenzia: number)`	White box
3	`createOfferta`	`OffertaDAO`	`(idImmobile: number, idUtente: number, prezzoOfferto: number)`	White box
4	`getNearbyPlaces`	`utils/geoapify`	`(lat: number, lon: number, radius?: number)`	White box

Tabella 3: Metodi testati come riportati in Test-Doc.

2.6.3 Dettaglio metodo 1: `updateStatoOfferta`

Non banalita. Esegue una UPDATE parametrizzata e rimappa il risultato in `OffertaDTO` validato con Zod (inclusa conversione `prezzoOfferto` stringa -> number).

Classe	Input rappresentativo	Output atteso
CE1 – stato valido, offerta esistente	`idOfferta=42, nuovoStato='Accettata'`	DTO con `stato='Accettata'` e `prezzoOfferto=200000` (number)
CE2 – offerta inesistente	`rows: []` dal mock	`mapRowToOfferta(undefined)` con eccezione Zod (gestione controller, fuori scope unit)

Classi di equivalenza (input/output).

Criteri e test case. Statement coverage; verifica conversione `'200000' -> 200000`. Test case: query UPDATE con parametri `['Accettata', 42]`, verifica `stato`, `idOfferta`, `prezzoOfferto`.

2.6.4 Dettaglio metodo 2: `assignUserToAgency`

Non banalita. UPDATE con clausola ruolo `Ruolo IN ('Agente', 'Supporto')`, mapping in `UtenteDTO`, restituzione `null` se non assegnabile.

Classe	Input rappresentativo	Output atteso
CE1 – utente Agente/Supporto assegnato	`idUtente=7, idAgenzia=3`, ruolo='Agente'	UPDATE con ruolo consentito, DTO con `idAgenzia=3`
CE2 – utente non esiste/non consentito	`idUtente=999, idAgenzia=3`, `rows: []`	`pool.query` vuoto, funzione ritorna `null`

Classi di equivalenza (input/output).

Criteri e test case. Statement + branch coverage su `result.rows[0] ? ... : null`. Test cases: (1) assegnazione riuscita con parametri `[3, 7]`; (2) utente non assegnabile con ritorno `null`.

2.6.5 Dettaglio metodo 3: `createOfferta`

Non banalita. Delega a `insertOfferta` con `offertaManuale=false` e `idOffertaOriginale=null`, stato iniziale `'InAttesa'`, mapping DTO validato.

Classe	Input rappresentativo	Output atteso
CE1 – offerta standard senza controproposta	`idImmobile=50, idUtente=20, prezzoOfferto=90000`	INSERT con `[50, 20, 90000, false, null]`, `offertaManuale=false`, `stato='InAttesa'`
CE2 – offerta con `offertaOriginaleId` (path `createManualOfferta`)	Non testata in questa suite	Copertura prevista a livello integrazione controller

Classi di equivalenza (input/output).

Criteri e test case. Statement coverage su `createOfferta`; verifica di `offertaOriginaleId || null` in `insertOfferta`. Test case: verifica parametri INSERT, `idOfferta`, `prezzoOfferto` numerico, `offertaManuale=false`.

2.6.6 Dettaglio metodo 4: `getNearbyPlaces`

Non banalita. Lettura chiave API a runtime, chiamate a Geoapify su 3 categorie fisse (`education.school`, `leisure.park`, `public_transport`), aggregazione risultati, gestione best-effort degli errori HTTP per categoria.

Classe	Input rappresentativo	Output atteso
CE1 – chiave presente, API risponde	`lat=45.0, lon=9.0, radius=500`; mock 1+0+1 feature	Lista di 2 `Place`, 3 chiamate `axios.get`
CE2 – chiave assente	`delete process.env.GEOAPIFY_KEY`	Eccezione `'Chiave Geoapify mancante'` prima di chiamate HTTP
CE3 – errore HTTP su una categoria	seconda call rigettata	Aggregazione delle altre categorie senza throw

Classi di equivalenza (input/output).

Criteri e test case. Branch coverage su ramo `!GEOAPIFY_KEY`, ciclo `for...of` (3 iterazioni), blocco `catch`. Boundary value su `radius` default 1000 vs esplicito 500. Test cases: percorso nominale (2 risultati) e chiave mancante (eccezione).

2.6.7 Riepilogo criteri strutturali (tabella Test-Doc)

Test	Statement	Branch	Condition	Boundary value
`updateStatoOfferta`	Si	–	–	`idOfferta` intero positivo, `prezzoOfferto` float->number
`assignUserToAgency`	Si	Si (rows vs null)	Si (check `result.rows[0]`)	ruolo consentito vs non consentito
`createOfferta`	Si	–	Si (`offertaOriginaleId || null`)	`prezzoOfferto` esatto
`getNearbyPlaces`	Si	Si (!KEY, ciclo, catch)	Si (`!GEOAPIFY_KEY`)	`radius` default 1000 vs 500

Tabella 4: Riepilogo criteri come da Test-Doc.

2.6.8 File e configurazione

File	Scopo
`dietiestates-backend/test/backend.test.ts`	Suite Jest con 4 describe e 6 test case
`dietiestates-backend/jest.config.ts`	Configurazione Jest (`ts-jest`, ambiente `node`)
`dietiestates-backend/package.json`	Script test: `jest -runInBand -detectOpenHandles`

Tabella 5: File di riferimento riportati in Test-Doc.

2.7 Esito dell'esecuzione reale della suite

L'esecuzione della suite backend ha prodotto i seguenti risultati:

- 1 suite eseguita con esito positivo;
- 6 test superati su 6;
- nessun fallimento e nessun test skipped;
- coverage complessiva pari a 70.9% sulle statement, 54.54% sui branch, 39.28% sulle funzioni e 64.44% sulle linee.

La Tabella 6 riporta i valori principali emersi dalla run dei test.

Ambito	Statement	Branch	Functions	Lines
Coverage complessiva backend	70.9%	54.54%	39.28%	64.44%

Tabella 6: Coverage effettivamente rilevata dalla suite Jest backend.

Questi valori mostrano che la suite attuale e adeguata come base di verifica dei metodi piu sensibili selezionati per la consegna, pur lasciando margine per estendere la copertura verso controller, middleware e casi di errore ulteriori.

3 Valutazione dell'usabilita

3.1 Obiettivi e collegamento ai requisiti

La valutazione dell'usabilita e stata definita in coerenza con i requisiti non funzionali presenti nella documentazione di progetto: interfaccia intuitiva, accessibilita multi-dispositivo, supporto a utenti finali senza training specifico e feedback informativi in caso di errore. I flussi presi come riferimento sono soprattutto quelli descritti in UC2, relativo al caricamento immobili, e UC4, relativo alla gestione delle offerte.

Gli obiettivi della valutazione sono:

- verificare l'efficacia dei flussi principali per utenti con ruoli differenti;
- individuare difetti di interazione ricorrenti prima della consegna;
- proporre correzioni concrete e prioritarie a partire da evidenze osservabili.

3.2 Expert review / inspection

3.2.1 Metodo adottato

La expert review e stata organizzata come ispezione sistematica basata sulle dieci euristiche di Nielsen, adattate al dominio del portale immobiliare. L'analisi ha coperto i principali percorsi applicativi: autenticazione, ricerca immobili, caricamento di un nuovo immobile, gestione offerte e consultazione dello storico.

3.2.2 Checklist di review

La Tabella 7 riporta la checklist applicata al prodotto finito.

Codice	Criterio	Esito	Osservazioni
A1	Il sistema mostra feedback durante operazioni asincrone	Soddisfatto	Spinner e messaggi di caricamento presenti
A2	Le operazioni di inserimento e aggiornamento producono feedback chiari	Soddisfatto	Successi ed errori comunicati nelle form principali
B1	Terminologia coerente con il dominio immobiliare	Soddisfatto	Lessico uniforme tra frontend e documentazione
C1	L'utente mantiene il controllo del flusso di navigazione	Soddisfatto	Percorsi principali navigabili senza dead-end
D1	Componenti e layout sono consistenti tra le pagine	Soddisfatto	Riutilizzo di navbar, card e pulsanti
E1	Sono presenti validazioni lato client e lato server	Soddisfatto	Form e backend controllano input e vincoli di dominio

Codice	Criterio	Esito	Osservazioni
E2	I messaggi di errore permettono di capire subito cosa correggere	Parzialmente soddisfatto	In alcuni casi il campo errato non e evidenziato abbastanza
F1	I filtri di ricerca sono facilmente comprensibili	Soddisfatto	Flusso di ricerca lineare e riconoscibile
F2	I filtri attivi restano visibili dopo l'applicazione	Parzialmente soddisfatto	Opportuno aggiungere badge o evidenza persistente
G1	I diversi ruoli hanno viste coerenti con i rispettivi compiti	Soddisfatto	Dashboard e pagine differenziate per ruolo
H1	L'interfaccia evita sovraccarico informativo	Soddisfatto	Struttura visiva pulita nelle pagine principali
I1	Il sistema aiuta il recupero dagli errori	Parzialmente soddisfatto	Migliorabile la localizzazione degli errori in input complessi
L1	E presente aiuto contestuale o documentazione d'uso integrata	Non soddisfatto	Mancano FAQ o tooltip strutturati

Tabella 7: Checklist di expert review applicata al prodotto finito.

3.2.3 Esiti e priorita di intervento

L'ispezione evidenzia un buon livello generale di usabilita, soprattutto nei flussi di consultazione e nelle operazioni guidate. Le criticita principali individuate sono le seguenti:

1. evidenziare in modo piu esplicito il campo che genera un errore di validazione;
2. rendere persistente la visibilita dei filtri di ricerca attivi;
3. introdurre aiuto contestuale minimo nelle pagine con maggiore densita funzionale.

3.3 Esperimento con utenti reali o potenziali

3.3.1 Soggetti reclutati

Per la valutazione sperimentale e stato definito un campione di cinque soggetti reclutati per convenienza ma differenziati per profilo d'uso:

- 2 agenti immobiliari, utenti esperti del dominio;
- 2 amministratori o gestori di agenzia, utenti avanzati di software gestionali;
- 1 cliente potenziale, rappresentativo dell'utente finale non esperto.

I partecipanti utilizzano abitualmente applicazioni web, ma non hanno preso parte allo sviluppo del sistema.

3.3.2 Procedura sperimentale

La procedura sperimentale e stata articolata in quattro fasi:

1. briefing iniziale sul contesto della prova e sulla tecnica del think-aloud;
2. breve esplorazione libera del sistema per ridurre l'effetto novita;
3. esecuzione di task rappresentativi dei casi d'uso UC2 e UC4;
4. survey finale con raccolta di giudizi quantitativi e commenti qualitativi.

3.3.3 Task sperimentali

Task	Descrizione	Criterio di successo
T1	Registrazione e login con credenziali tradizionali	Accesso completato senza assistenza
T2	Ricerca immobili con almeno due filtri	Visualizzazione dei risultati attesi
T3	Apertura del dettaglio immobile con consultazione mappa	Comprensione corretta della scheda e dei servizi vicini
T4	Inserimento di un'offerta su un immobile	Offerta inviata oppure flusso compreso senza blocchi
T5	Consultazione dello storico offerte	Storico raggiunto e interpretato correttamente

Tabella 8: Task usati nell'esperimento di usabilita.

3.3.4 Metriche osservate

Per ciascun task sono state raccolte le seguenti metriche:

- tasso di completamento;
- tempo medio di esecuzione;
- numero medio di errori o tentativi falliti;
- richieste di aiuto;
- valutazione soggettiva post-task e post-sessione.

3.3.5 Survey post-esperimento

Il survey e stato diviso in due parti: una scala SUS a 10 item e un gruppo di domande specifiche sul dominio applicativo.

Questionario SUS. Le affermazioni, valutate su scala Likert 1–5, sono le seguenti:

1. Penso che userei questo sistema frequentemente.
2. Ho trovato il sistema inutilmente complesso.
3. Ho trovato il sistema facile da usare.
4. Avrei bisogno del supporto di un tecnico per usarlo.
5. Ho trovato le varie funzioni del sistema ben integrate.
6. Ho trovato troppe inconsistenze nel sistema.
7. La maggior parte delle persone imparerebbe a usarlo rapidamente.
8. Ho trovato il sistema macchinoso da usare.
9. Mi sono sentito a mio agio nell'uso del sistema.
10. Ho dovuto imparare molte cose prima di usarlo bene.

Domande specifiche.

- La ricerca immobili con filtri e stata intuitiva?
- I messaggi di errore erano chiari e utili?
- La mappa geografica e risultata utile?
- Il processo di inserimento e gestione offerte e apparso logico?
- Quali funzionalita sono risultate piu difficoltose?
- Quali miglioramenti suggeriresti?

3.3.6 Risultati

I risultati quantitativi principali sono riportati nella Tabella 9.

Task	Completamento	Tempo medio	Errori medi
T1 – Registrazione/login	100%	2.1 min	0.4
T2 – Ricerca con filtri	80%	3.5 min	1.2
T3 – Dettaglio + mappa	100%	2.8 min	0.2
T4 – Inserimento offerta	80%	4.1 min	1.6
T5 – Storico offerte	100%	1.9 min	0.0

Tabella 9: Risultati sintetici dell'esperimento di usabilita.

Il punteggio SUS medio rilevato e pari a **71.5/100**, cioe superiore alla soglia comunemente considerata accettabile. Le medie delle domande specifiche risultano pari a 4.2/5 per la ricerca con filtri, 3.4/5 per la chiarezza dei messaggi di errore, 4.0/5 per l'utilita della mappa e 3.6/5 per la logica del flusso di offerta.

3.3.7 Discussione dei risultati

I dati raccolti mostrano che i flussi di autenticazione, consultazione del dettaglio e storico offerte risultano solidi, con completamento pieno e pochi errori. Le criticita si concentrano invece sulle interazioni che richiedono piu passi decisionali o un feedback piu esplicito, in particolare:

- la visibilita del pulsante o del flusso di inserimento offerta;
- la chiarezza dei messaggi di validazione;
- la riconoscibilita persistente dei filtri applicati alla ricerca.

Nel complesso il sistema si colloca in una fascia di usabilita buona, ma con margini di miglioramento ancora chiari e circoscritti. Le evidenze dell'esperimento confermano inoltre le criticita gia emerse nella expert review, rafforzando la priorita degli interventi suggeriti.

4 Documentazione del processo di sviluppo

4.1 Versionamento

Particolare attenzione e stata dedicata all'impiego di strumenti di controllo versione come **Git** e **GitHub**, che hanno permesso di mantenere una cronologia completa e dettagliata delle modifiche apportate, facilitando il lavoro collaborativo e garantendo una gestione efficace delle versioni. Tali strumenti hanno reso possibile la generazione di report statistici utili a documentare in modo oggettivo l'impegno del gruppo: numero di commit, distribuzione dei contributi e frequenza degli aggiornamenti costituiscono indicatori significativi della progressione dello sviluppo.

4.1.1 GitHub

Durante lo sviluppo del progetto e stato adottato un flusso di lavoro basato su Git e GitHub, con l'obiettivo di garantire un processo collaborativo efficace, ordinato e tracciabile.

La repository ufficiale del progetto e disponibile pubblicamente al seguente indirizzo:

`https://github.com/Kenobi1797/INGSW-DietiEstates25`

Il ciclo di lavoro ha previsto le seguenti operazioni:

1. **Commit frequenti** per tracciare ogni modifica significativa, accompagnati da messaggi descrittivi e coerenti con le funzionalita sviluppate.
2. **Push regolari** verso il repository remoto, in modo da sincronizzare il lavoro e mantenere sempre aggiornata la cronologia condivisa.
3. **Merge** verso il branch principale (`main`) effettuato solo dopo verifica dell'assenza di conflitti e del corretto funzionamento delle funzionalita integrate.

File .gitignore. Nel progetto e stato definito un file `.gitignore` per escludere dal versionamento file e cartelle non pertinenti al codice sorgente, come dipendenze, artefatti di compilazione, file di ambiente e upload generati a runtime. La presenza di questo file e fondamentale per evitare che credenziali sensibili (come variabili d'ambiente contenenti chiavi API, segreti JWT e credenziali del database) vengano accidentalmente incluse nel repository remoto. In particolare sono esclusi:

- `node_modules/` – dipendenze npm riproducibili tramite `npm install`;

- `dist/` – artefatti di compilazione TypeScript;
- `.env`, `.env.*` – file di configurazione con segreti (viene versionato solo `.env.example` con valori placeholder);
- `uploads/*` – file caricati a runtime (immagini degli immobili), con eccezione del file `.gitkeep` per preservare la struttura della directory;
- `coverage/` – report di copertura generati dalla suite Jest.

4.1.2 Statistiche del repository

Le statistiche principali rilevate al momento della consegna sono riportate nella Tabella 10.

Metrica	Valore
Numero totale di commit	158
Numero di contributors	2
Branch principale	`main`
Data del primo commit	26 agosto 2025
Data dell'ultimo commit	30 marzo 2026
Durata complessiva del progetto	circa 7 mesi

Tabella 10: Statistiche generali del repository Git.

Distribuzione dei commit per contributor. Il repository conta due contributors. La Tabella 11 riporta la distribuzione dei commit per autore:

Contributor	Identita Git	Commit
Membro principale	`kenobi1797`	145
Membro secondario	`pietrodok`	13
Totale		**158**

Tabella 11: Distribuzione dei commit per contributor.

Frequenza dei commit per settimana. La Tabella 12 riporta la distribuzione settimanale dei commit secondo i dati di *GitHub Insights* (https://github.com/Kenobi1797/INGSW-DietiEstates25/graphs/commit-activity), che registra solo le settimane con almeno un commit. Si individuano tre fasi distinte: una fase di avvio (agosto–ottobre 2025) con attivita regolare, un lungo periodo di inattivita quasi totale (novembre 2025 – inizio marzo 2026) e uno sprint finale concentrato nelle ultime due settimane di marzo 2026, che da soli rappresentano il 60% dell'intera attivita.

Settimana (domenica iniziale)	Periodo	Commit
24 agosto 2025	Fase 1 – Avvio	5
7 settembre 2025	Fase 1 – Avvio	1
14 settembre 2025	Fase 1 – Avvio	6
21 settembre 2025	Fase 1 – Avvio	4
28 settembre 2025	Fase 1 – Avvio	14
5 ottobre 2025	Fase 1 – Avvio	19
12 ottobre 2025	Fase 1 – Avvio	5
9 novembre 2025	Fase 2 – Intermittente	2
7 dicembre 2025	Fase 2 – Intermittente	1
21 dicembre 2025	Fase 2 – Intermittente	1
18 gennaio 2026	Fase 2 – Intermittente	1
15 marzo 2026	Fase 3 – Sprint finale	3
22 marzo 2026	Fase 3 – Sprint finale	**78**
29 marzo 2026	Fase 3 – Sprint finale	14
Totale (settimane con attivita)		**154**

Tabella 12: Distribuzione settimanale dei commit secondo GitHub Insights (settimane con commit > 0).

Il picco massimo si registra nella settimana del 22 marzo 2026, con **78 commit** in sette giorni, corrispondente alla fase di completamento delle funzionalita, integrazione dei test e revisione finale della documentazione prima della consegna. La settimana successiva (29 marzo) aggiunge ulteriori 14 commit di rifinitura.

4.2 Codice sorgente e containerizzazione

Il codice sorgente del progetto e organizzato come monorepo nel repository Git, con le seguenti directory principali:

- `dietiestates-backend/` – API REST Node.js/TypeScript (Express, PostgreSQL);
- `dietiestates-frontend/` – applicazione web Next.js/TypeScript;
- `Progettazione/` – requisiti, glossario, casi d'uso e documentazione dei test;
- `Documentazione/` – documenti di consegna in formato LaTeX.

Per garantire portabilita, isolamento dell'ambiente e riproducibilita della build, l'intero stack e stato containerizzato tramite **Docker** e orchestrato con **Docker Compose**.

4.2.1 Dockerfile backend

L'immagine del backend viene costruita a partire da `node:22-alpine`, scelta per la dimensione ridotta e la minore superficie d'attacco rispetto alle immagini full Debian. Le dipendenze vengono installate separatamente dal codice sorgente per sfruttare la cache dei layer Docker:

```
COPY package*.json ./
RUN npm prune && npm install && npm dedupe && npm audit fix
```

L'esecuzione di `npm audit fix` durante la build consente di rilevare e correggere automaticamente eventuali vulnerabilita note nelle dipendenze. Successivamente vengono copiati i sorgenti e la configurazione:

```
COPY tsconfig.json jest.config.ts ./
COPY src ./src
COPY test ./test
```

Per rispettare il principio del minimo privilegio, il processo viene eseguito come utente non-root:

```
RUN addgroup -S appgroup && adduser -S appuser -G appgroup \
    && chown -R appuser:appgroup /usr/src/app
USER appuser

EXPOSE 5000
CMD ["npm", "run", "dev"]
```

4.2.2 Dockerfile frontend

Il frontend adotta anch'esso `node:22-alpine` e utilizza `npm ci` al posto di `npm install` per garantire installazioni riproducibili e deterministiche, basate sul `package-lock.json`:

```
FROM node:22-alpine AS dev
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm ci && npm cache clean --force

COPY next-env.d.ts next.config.ts postcss.config.mjs \
     eslint.config.mjs tsconfig.json ./
COPY public ./public
COPY src ./src

RUN chown -R node:node /usr/src/app
USER node
EXPOSE 3000
```

4.2.3 docker-compose.yml e file di build automatica

L'orchestrazione dell'intero stack e gestita dal file `docker-compose.yml` presente nella radice del repository. Esso definisce tre servizi con dipendenze di avvio verificate tramite healthcheck:

- **dietiestates-db** – PostgreSQL 17, healthcheck tramite `pg_isready`;
- **dietiestates-backend** – esposto sulla porta 5000, avvio condizionato all'healthcheck del database;
- **dietiestates-frontend** – esposto sulla porta 3000, avvio condizionato all'healthcheck del backend.

Le variabili d'ambiente sensibili (credenziali database, JWT secret, OAuth credentials) sono lette dal file `.env` tramite la direttiva `env_file`, in modo da non essere incluse nel file `docker-compose.yml` versionato. L'intero stack puo essere avviato con un unico comando:

```
docker compose up --build
```

Gli script di build definiti nei rispettivi `package.json` sono riportati nella Tabella 13.

Componente	Script	Descrizione
Backend	`npm run build`	Compilazione TypeScript con `tsc` (output in `dist/`)
Backend	`npm run dev`	Avvio con `nodemon` + `ts-node` (hot reload in sviluppo)
Backend	`npm test`	Suite Jest con `-runInBand -detectOpenHandles`
Frontend	`npm run build`	Build di produzione Next.js (`next build`)
Frontend	`npm run dev`	Server di sviluppo Next.js con hot reload
Frontend	`npm run lint`	Analisi statica del codice con ESLint

Tabella 13: Script di build e sviluppo definiti nei `package.json`.

5 Conclusioni

L'analisi dei file di test e della documentazione di progetto mostra che DietiEstates25 dispone gia di una suite xUnit coerente per quattro metodi backend non banali, progettata con criteri di equivalenza, copertura strutturale e isolamento delle dipendenze esterne. La copertura ottenuta e sufficiente per documentare la strategia adottata e per dimostrare una base di verifica effettiva e funzionante.

Dal punto di vista dell'usabilita, la combinazione di expert review e sperimentazione con utenti fornisce una valutazione credibile e completa: i flussi principali risultano efficaci, mentre le aree di miglioramento riguardano soprattutto visibilita dei filtri attivi, messaggi di validazione e supporto contestuale.

Il progetto dimostra inoltre un utilizzo coerente degli strumenti di sviluppo moderni: containerizzazione con Docker, gestione delle dipendenze tramite npm, build riproducibili e controllo di versione distribuito con Git e GitHub, con una storia di commit che copre l'intero ciclo di vita del progetto.